

Ex-no: 8	PRODUCER AND CONSUMER USING SYNCRONISATION
17/03/2026	

Aim

To study and implement the Producer–Consumer problem using synchronization techniques to ensure proper sharing of a common buffer without data inconsistency.

Algorithm

1. Initialize:
 - A fixed-size buffer.
 - Two counting variables: `empty` (number of empty slots) and `full` (number of filled slots).
 - A mutual exclusion lock (mutex).
2. **Producer Process:**
 - Wait until there is an empty slot (`empty > 0`).
 - Acquire the mutex lock.
 - Produce an item and insert it into the buffer.
 - Release the mutex lock.
 - Update counters: decrement `empty`, increment `full`.
3. **Consumer Process:**
 - Wait until there is a filled slot (`full > 0`).
 - Acquire the mutex lock.
 - Remove an item from the buffer.
 - Release the mutex lock.
 - Update counters: increment `empty`, decrement `full`.
4. Repeat the producer and consumer operations continuously.

CODE:

```
// Producer Consumer
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int mutex = 1, full = 0, empty = 5, x = 0;
```

```
int wait(int s)
```

```
{
```

```
    return --s;
```

```
}
```

```
int signal(int s)
```

```
{
```

```
    return ++s;
```

```
}
```

```
void producer()
```

```
{
```

```
    mutex = wait(mutex);
```

```
    empty = wait(empty);
```

```
    full = signal(full);
```

```
    x++;
```

```
    printf("\nProducer produces item %d", x);
```

```
    mutex = signal(mutex);
```

```
}
```

```
void consumer()
```

```
{  
  
    mutex = wait(mutex);  
  
  
    full = wait(full);  
  
    empty = signal(empty);  
  
  
    printf("\nConsumer consumes item %d", x);  
  
    x--;  
  
  
    mutex = signal(mutex);  
}
```

```
int main()  
  
{  
  
    int n;  
  
  
  
    printf("\n----- Producer Consumer Problem -----");  
    printf("\n1. Producer");  
    printf("\n2. Consumer");  
    printf("\n3. Exit");  
  
  
    while(1)  
    {  
  
        printf("\nEnter your choice: ");  
  
        scanf("%d",&n);
```

```
switch(n)
{
    case 1:
        if(mutex==1 && empty!=0)
        {
            producer();
        }
        else
        {
            printf("\nBuffer is full!! Producer cannot produce.");
        }
        break;

    case 2:
        if(mutex==1 && full!=0)
        {
            consumer();
        }
        else
        {
            printf("\nBuffer is empty!! Consumer cannot consume.");
        }
        break;
```

```

        case 3:

            printf("\nExiting program...");

            printf("\nFinal buffer items produced: %d\n", x);

            exit(0);


        default:

            printf("\nInvalid choice. Try again.");

    }

}

return 0;

}

```

OUTPUT:

Case 1: Producer Operation

```

Enter your choice: 1

Producer produces item 1
Enter your choice: 1

Producer produces item 2

```

Case 2: Consumer Operation

```

Enter your choice: 2

Consumer consumes item 2
Enter your choice: 2

Consumer consumes item 1

```

Case 3: Buffer Full Condition

```

Buffer is full!! Producer cannot produce.

```

Case 4: Buffer Empty Condition

```

Buffer is empty!! Consumer cannot consume.

```

Case 5: Exit

Enter your choice: 3

Exiting program...

Final buffer items produced: 0

Result

The Producer–Consumer problem was successfully implemented using synchronization mechanisms. Proper coordination between producer and consumer processes was achieved, ensuring no data loss, buffer overflow, or underflow.